



**Laporan Praktikum Algoritma & Pemrograman
Semester Genap 2025/2026**

SAYA MENYATAKAN BAHWA LAPORAN PRAKTIKUM INI SAYA BUAT DENGAN USAHA SENDIRI TANPA MENGGUNAKAN BANTUAN ORANG LAIN. SEMUA MATERI YANG SAYA AMBIL DARI SUMBER LAIN SUDAH SAYA CANTUMKAN SUMBERNYA DAN TELAH SAYA TULIS ULANG DENGAN BAHASA SAYA SENDIRI.

SAYA SANGGUP MENERIMA SANKSI JIKA MELAKUKAN KEGIATAN PLAGIASI, TERMASUK SANKSI TIDAK LULUS MATA KULIAH INI.

NIM	71251233
Nama Lengkap	Mikael Gratianus Satrio Adi Kuncoro
Minggu ke / Materi	13 / Tipe Data Tuples

**PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS KRISTEN DUTA WACANA
YOGYAKARTA
2026**

Tuple Immutable

Tuple mirip dengan list karena dapat menyimpan berbagai jenis nilai dan memiliki indeks integer. Bedanya, tuple bersifat immutable sehingga isinya tidak dapat diubah. Tuple juga bisa dibandingkan dan bersifat hashable, sehingga dapat digunakan sebagai key pada dictionary Python.

Objek hashable adalah objek yang memiliki nilai hash tetap dan dapat dibandingkan dengan objek lain. Objek ini dapat digunakan sebagai key pada dictionary dan anggota set. Sebagian besar objek bawaan Python bersifat hashable, sedangkan objek yang dapat diubah seperti list dan dictionary tidak hashable.

Berikut beberapa sintaks penulisan tuple pada python:

```
1 # Sintaks tuple
2 t = 'a','b','c','d','e'
3
4 # Sintaks tuple lainnya
5 t = ('a','b','c','d','e')
6
7 # Tuple satu element, ditambahkan koma ( , ) dibelakang penulisan tuple
8 t1 = ('a',)
9 tipe = type(t1)
10 print(tipe) # Outputnya: <class 'tuple'>
11
12 # Jika tidak menambahkan koma hasilnya akan string
13 t2 = ('a')
14 tipe2 = type(t2)
15 print(tipe2) # Outputnya: <class 'str'>
16
17 # Model lain jika menggunakan fungsi built-in tuple, jika tidak diisi dengan argument apapun akan membentuk tuple kosong
18 t = tuple()
19 print(t) # Outputnya: ()
20
21 # Jika argumennya berupa urutan (string, list, atau tuple), akan mengembalikan nilai tuple dengan elemen-elemen yang berurutan
22 t = tuple('dutawacana')
23 print(t) # Outputnya: ('d','u','t','a','w','a','c','a','n','a')
24
25 # Tuple adalah nama dari constructor, tidak bisa digunakan sebagai nama variabel. operator yang bekekerja pada list juga kerja pada tuple
26 # tanda ini menandakan indeks element dari tuple
27 t = ('a', 'b', 'c', 'd', 'e')
28 print(t[0]) # Outputnya: 'a'
29 print(t[1:3]) # Untuk menampilkan rentang nilai dari element tuple, outputnya: ('b', 'c')
30 ubah = t[0] = 'A'
31 print(ubah) # TypeError: object doesn't support item assignment
32 ubah2 = ('A',) + t[1:]
33 print(ubah2) # Element pada tuple tidak dapat dirubah tetapi dapat diganti dengan elemen lain. Outputnya: ('A', 'b', 'c', 'd', 'e')
```

Gambar 1.1: Penulisan tuple pada python

Membandingkan Tuple

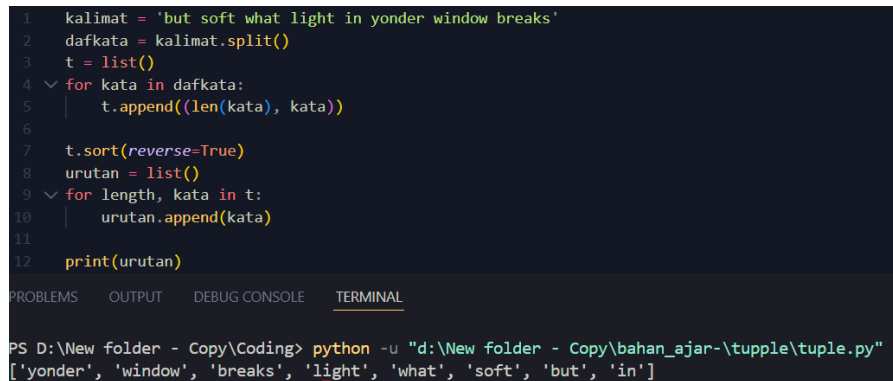
Operator perbandingan dapat digunakan pada tuple dan struktur sekuensial lain seperti list, dictionary, dan set. Perbandingan dilakukan dengan mengecek elemen pertama, lalu berlanjut ke elemen berikutnya jika masih sama, hingga ditemukan perbedaan. Berikut contoh penulisan nya;

```
>>> (0, 1, 2) < (0, 3, 4)
True
>>> (0, 1, 2000000) < (0, 3, 4)
True
>>>
```

Gambar 1.2: Penulisan tuple dengan operator perbandingan

Fungsi sort pada Python bekerja dengan mengurutkan berdasarkan elemen pertama terlebih dahulu, lalu elemen berikutnya jika diperlukan. Metode ini dikenal sebagai DSU yaitu **decorate** untuk membuat daftar tuple dengan key pengurutan sebelum elemen utama, **sort** untuk mengurutkan list tuple menggunakan fungsi sort, **undecorate** untuk mengambil kembali elemen yang sudah diurutkan ke dalam satu sekuensial.

Berikut contoh penulisan kode nya dengan sebuah kalimat lalu akan melakukan pengurutan kata dalam kalimat tersebut dari yang paling panjang ke yang paling pendek.



```
1 kalimat = 'but soft what light in yonder window breaks'
2 dafkata = kalimat.split()
3 t = list()
4 for kata in dafkata:
5     t.append((len(kata), kata))
6
7 t.sort(reverse=True)
8 urutan = list()
9 for length, kata in t:
10     urutan.append(kata)
11
12 print(urutan)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

PS D:\New folder - Copy\Coding> python -u "d:\New folder - Copy\bahan_ajar-\tuple\tuple.py"

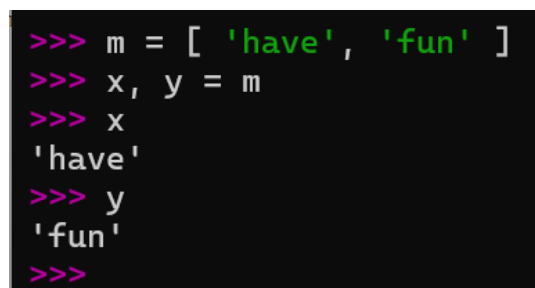
['yonder', 'window', 'breaks', 'light', 'what', 'soft', 'but', 'in']

Gambar 1.3: Kode pengurutan kata dari panjang ke pendek

Looping pertama membuat daftar tuple berisi kata dan panjangnya. Fungsi sort mengurutkan berdasarkan elemen pertama, lalu elemen berikutnya jika diperlukan. Keyword **reverse=True** digunakan untuk mengurutkan secara menurun (descending). Looping kedua membuat daftar tuple yang diurutkan berdasarkan panjang kata dan alfabet. Pada contoh, empat kata diurutkan secara alfabet terbalik sehingga kata “what” muncul sebelum “soft” dalam list. Kata-kata yang muncul diurutkan sebagai list dengan urutan panjang kata dari yang paling panjang ke kata yang paling pendek.

Penugasan Tuple

Python memiliki fitur unik yang memungkinkan penggunaan tuple di sisi kiri assignment. Fitur ini memungkinkan beberapa variabel diisi sekaligus secara berurutan dalam satu statement, misalnya menetapkan elemen pertama dan kedua ke variabel x dan y.



```
>>> m = [ 'have', 'fun' ]
>>> x, y = m
>>> x
'have'
>>> y
'fun'
>>>
```

Gambar 1.4: Penetapan element pada tuple

Python akan menterjemahkan sintaks tuple dalam langkah-langkah sebagai berikut:

```
>>> m = [ 'have', 'fun' ]
>>> x = m[0]
>>> y = m[1]
>>> x
'have'
>>> y
'fun'
>>>
```

Gambar 1.5: Gambaran python menterjemahkan sintaks tuple

Jika menggunakan tanda kurung (parentheses), sebagai berikut:

```
>>> m = [ 'have', 'fun' ]
>>> (x, y) = m
>>> x
'have'
>>> y
'fun'
>>>
```

Gambar 1.6: Penetapan element pada tuple dengan tanda ()

Tuple memungkinkan pula untuk menukar nilai variabel dalam satu statement.

Berikut contohnya:

```
>>>> a, b = b, a
```

Pada contoh tersebut, kedua statement adalah tuple. Nilai pada tuple sebelah kanan akan ditugaskan ke variabel pada tuple sebelah kiri secara berurutan, dan semua ekspresi di sebelah kanan dievaluasi terlebih dahulu sebelum penugasan dilakukan.

Yang perlu diperhatikan adalah jumlah variabel antara sisi kiri dan sisi kanan harus sama. Berikut contoh nya:

```
>>> a, b = 1, 2, 3
Traceback (most recent call last):
  File "<python-input-0>", line 1, in <module>
    a, b = 1, 2, 3
    ^^^
ValueError: too many values to unpack (expected 2, got 3)
>>>
```

Gambar 1.7: Penerapan variabel yang tidak sama jumlah nya sisi kanan dan kiri

Pada sisi sebelah kanan biasanya terdapat data sekuensial string, list atau tuple. Sebagai contoh, pembagian alamat email menjadi user name dan domain seperti berikut ini.

```

>>> email = 'kocakgaming@gmail.com'
>>> username, domain = email.split('@')
>>> print(username)
kocakgaming
>>> print(domain)
gmail.com
>>>

```

Gambar 1.8: Contoh pembagian 2 elemen yang dipisahkan dengan menggunakan split
 Nilai yang dikembalikan dari split terdiri dari dua elemen yang dipisahkan tanda "@", elemen pertama berisi username dan elemen selanjutnya berisi domain.

Dictionaries and Tuple

Dictionaries memiliki metode items() yang mengembalikan daftar tuple berisi pasangan key dan value.

```

>>> d = {'a':10, 'b':1, 'c':22}
>>> t = list(d.items())
>>> print(t)
[('a', 10), ('b', 1), ('c', 22)]
>>>

```

Gambar 1.9: Penerapan metode item() pada dictionary yang mengembalikan daftar tuple
 Lihat Gambar 1.9. Pada Python 3.7 ke atas, dictionary mempertahankan urutan input, tetapi sort() tetap diperlukan jika ingin mengurutkan data berdasarkan key atau value tertentu. Misalnya jika memerlukan sort():

```

>>> d = {'b':1, 'a':10, 'c':22}
>>> t = list(d.items())
>>> print(t)
[('b', 1), ('a', 10), ('c', 22)]
>>> t.sort()
>>> print(t)
[('a', 10), ('b', 1), ('c', 22)]
>>>

```

Gambar 2.0: Pengurutan dengan metode sort

Multipenugasan dengan dictionaries

```

1  d = {'a':10, 'b':1, 'c':22}
2  for key, val in list(d.items()):
3      print(val, key)

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

```

PS D:\New folder - Copy\Coding> python -u "d:\New folder - Copy\bahan_ajar-\tuple\tuple.py"
10 a
1 b
22 c
PS D:\New folder - Copy\Coding>

```

Gambar 2.1: Kombinasi antara items, tuple assignment dan for

Pada looping tersebut digunakan dua variabel, yaitu key dan val, karena items() mengembalikan pasangan tuple (key, value). Setiap iterasi mengambil satu pasangan data dari dictionary, lalu print(val, key) menampilkan value terlebih dahulu kemudian key.

Isi dictionary dapat dicetak berdasarkan nilai pada setiap pasangan key-value. Caranya adalah membuat list tuple berisi (value, key) menggunakan method items(), lalu mengurutkannya berdasarkan value secara terbalik sebelum dicetak kembali.

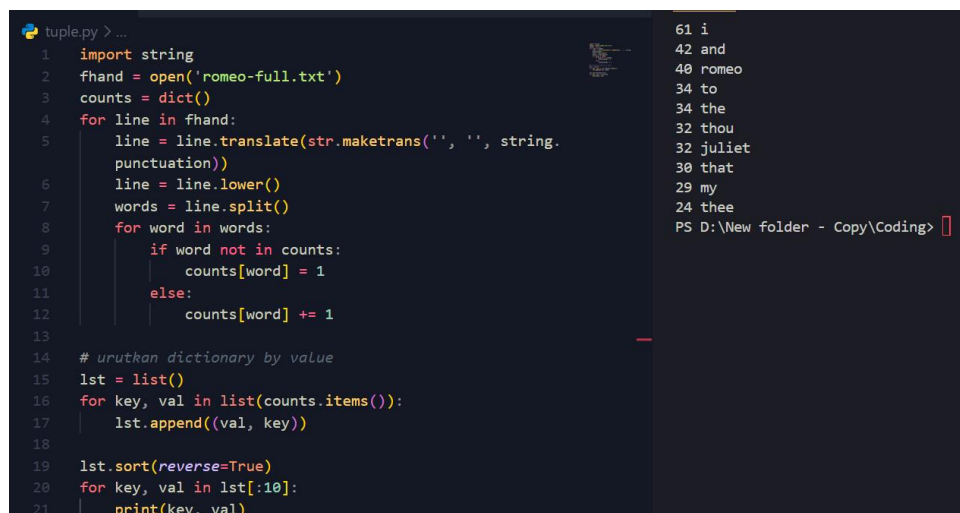
```
>>> d = {'a':10, 'b':1, 'c':22}
>>> l = list()
>>> for key, val in d.items() :
...     l.append( (val, key) )
...
>>> l
[(10, 'a'), (1, 'b'), (22, 'c')]
>>> l.sort(reverse=True)
>>> l
[(22, 'c'), (10, 'a'), (1, 'b')]
>>>
```

Gambar 2.2: Proses pengurutan dictionary berdasarkan value

Dengan menyusun list tuple yang menempatkan value sebagai elemen pertama, dictionary dapat diurutkan berdasarkan nilainya dengan lebih mudah.

Kata yang sering muncul

Pada kasus ini, akan dibuat program untuk menampilkan 10 kata yang paling sering muncul pada teks *Romeo and Juliet Act 2, Scene 2* menggunakan list dari tuple. File yang digunakan adalah romeo-full.txt.



```
tuple.py > ...
1 import string
2 fhand = open('romeo-full.txt')
3 counts = dict()
4 for line in fhand:
5     line = line.translate(str.maketrans('', '', string.
6         punctuation))
7     line = line.lower()
8     words = line.split()
9     for word in words:
10         if word not in counts:
11             counts[word] = 1
12         else:
13             counts[word] += 1
14
15 # urutkan dictionary by value
16 lst = list()
17 for key, val in list(counts.items()):
18     lst.append((val, key))
19
20 lst.sort(reverse=True)
21 for key, val in lst[:10]:
22     print(key, val)
```

```
61 i
42 and
40 romeo
34 to
34 the
32 thou
32 juliet
30 that
29 my
24 thee
PS D:\New folder - Copy\Coding>
```

Gambar 2.3: Program menghitung dan menampilkan 10 kata paling sering muncul pada teks *Romeo and Juliet* dengan dictionary dan list tuple.

Program diawali dengan membaca file dan menghitung frekuensi tiap kata menggunakan dictionary. Setelah itu dibuat list tuple (val, key) yang diurutkan secara menurun berdasarkan nilai. Jika nilainya sama, pengurutan dilakukan berdasarkan abjad key. Terakhir, program melakukan looping untuk menampilkan 10 kata yang paling sering muncul.

Tuple sebagai kunci dictionaries

Tuple bersifat hashable sehingga dapat digunakan sebagai composite key pada dictionary, sedangkan list tidak. Composite key dapat digunakan, misalnya, untuk memetakan pasangan nama belakang dan nama depan ke nomor telepon dalam dictionary.

```
directory[last,first] = number
```

Expression di dalam kurung merupakan tuple. Selanjutnya, tuple tersebut digunakan pada penugasan dalam perulangan for yang berhubungan dengan dictionary.

```
for last, first in directory:
```

```
    print(first, last, directory[last,first])
```

Looping pada key direktori menggunakan tuple. Setiap elemen tuple ditetapkan terlebih dahulu, lalu nama dan nomor telepon dicetak sesuai data.

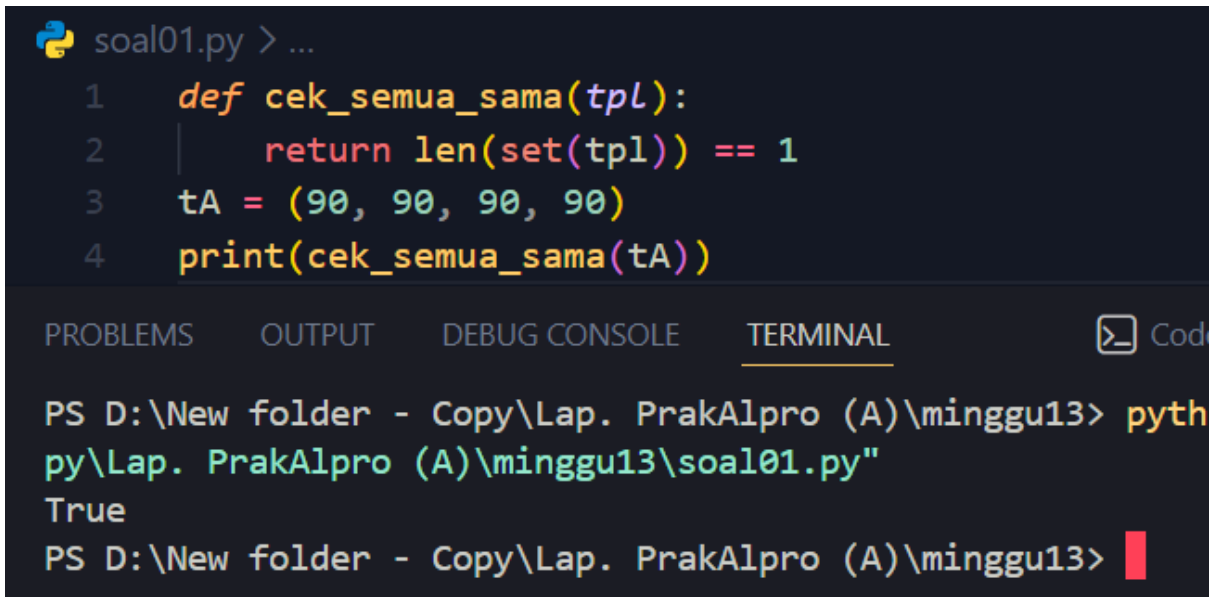
```
>>> last = 'gaming'
>>> first = 'kocak'
>>> number = '08223619719'
>>> direct = dict()
>>> direct[last, first] = number
>>> for last, first in direct:
...     print(first, last, direct[last, first])
...
kocak gaming 08223619719
```

Gambar 2.4: Implementasi dictionary untuk menyimpan nama dan nomor telepon.

LINK GITHUB

: <https://github.com/Rio-code-07/71251233-Rio-PrakAlpro.git>

SOAL 01



```
soal01.py > ...
1  def cek_semua_sama(tpl):
2      return len(set(tpl)) == 1
3  tA = (90, 90, 90, 90)
4  print(cek_semua_sama(tA))
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL Cod

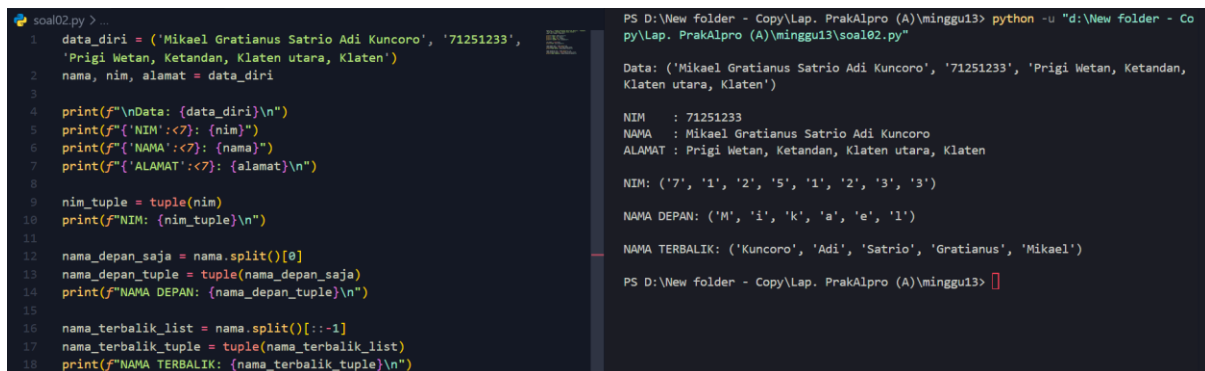
```
PS D:\New folder - Copy\Lap. PrakAlpro (A)\minggu13> pyth
py\Lap. PrakAlpro (A)\minggu13\soal01.py"
True
PS D:\New folder - Copy\Lap. PrakAlpro (A)\minggu13> 
```

Gambar 2.5: Source code soal 01

Penjelasan:

Ini adalah program untuk melakukan pengecekan apakah semua anggota yang ada didalam tuple sama. Pertama fungsi **cek_semua_sama(tpl)** didefinisikan untuk memeriksa apakah semua elemen dalam sebuah tuple memiliki nilai yang sama. Fungsi ini menerima satu parameter yaitu **tpl**, lalu pada baris **return len(set(tpl)) == 1** terjadi tiga proses, yaitu **set(tpl)** mengubah tuple menjadi **set** sehingga elemen duplikat otomatis dihapus, misalnya (90, 90, 90, 90) menjadi {90}. Setelah itu **len(...)** digunakan untuk menghitung jumlah elemen pada set tersebut, sehingga {90} memiliki panjang 1. Kemudian **== 1** digunakan untuk mengecek apakah jumlah elemennya tepat satu, yang berarti semua nilai dalam tuple sama. Hasil pengecekan akan berupa **True** atau **False**. Selanjutnya dibuat tuple **tA = (90, 90, 90, 90)** yang berisi empat angka 90, lalu fungsi **cek_semua_sama(tA)** dipanggil menggunakan **print()**, sehingga menghasilkan output **True** karena semua elemen dalam tuple memiliki nilai yang sama.

SOAL 02



```
soal02.py > ...
1 data_diri = ('Mikael Gratianus Satrio Adi Kuncoro', '71251233',
2   'Prigi Wetan, Ketandan, Klaten utara, Klaten')
3   nama, nim, alamat = data_diri
4
5   print(f"\nData: {data_diri}\n")
6   print(f"NIM:<7>: {nim}")
7   print(f"NAMA:<7>: {nama}")
8   print(f"ALAMAT:<7>: {alamat}\n")
9
10  nim_tuple = tuple(nim)
11  print(f"NIM: {nim_tuple}\n")
12
13  nama_depan_saja = nama.split()[0]
14  nama_depan_tuple = tuple(nama_depan_saja)
15  print(f"NAMA DEPAN: {nama_depan_tuple}\n")
16
17  nama_terbalik_list = nama.split()[::-1]
18  nama_terbalik_tuple = tuple(nama_terbalik_list)
19  print(f"NAMA TERBALIK: {nama_terbalik_tuple}\n")

PS D:\New folder - Copy\Lap. PrakAlpro (A)\minggu13> python -u "d:\New folder - Co
py\Lap. PrakAlpro (A)\minggu13\soal02.py"

Data: ('Mikael Gratianus Satrio Adi Kuncoro', '71251233', 'Prigi Wetan, Ketandan,
Klaten utara, Klaten')

NIM      : 71251233
NAMA     : Mikael Gratianus Satrio Adi Kuncoro
ALAMAT   : Prigi Wetan, Ketandan, Klaten utara, Klaten

NIM: ('7', '1', '2', '5', '1', '2', '3', '3')

NAMA DEPAN: ('M', 'i', 'k', 'a', 'e', 'l')

NAMA TERBALIK: ('Kuncoro', 'Adi', 'Satrio', 'Gratianus', 'Mikael')

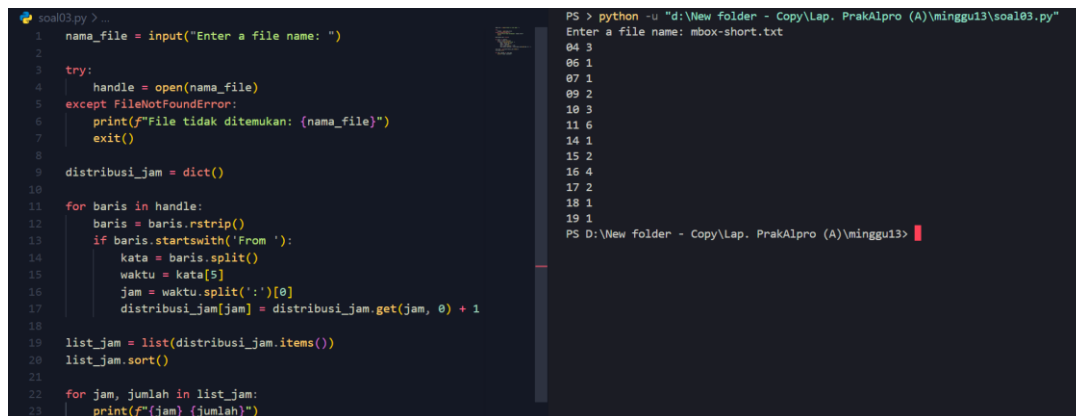
PS D:\New folder - Copy\Lap. PrakAlpro (A)\minggu13>
```

Gambar 2.6: Source code soal 02

Penjelasan:

Ini adalah program dengan menggunakan tuple yang dapat melakukan proses seperti pada kasus 11.1 pada materi tuple minggu 13. Pertama membuat tuple **data_diri** yang berisi tiga elemen, yaitu **nama lengkap**, **NIM**, dan **alamat**. Setelah itu dilakukan *tuple unpacking* pada baris **nama, nim, alamat = data_diri**, sehingga setiap elemen tuple langsung disimpan ke variabel **nama**, **nim**, dan **alamat**. Selanjutnya **print(f"\nData: {data_diri}\n")** digunakan untuk mencetak seluruh isi tuple ke layar dengan tambahan baris baru menggunakan **\n**. Baris berikutnya mencetak **NIM**, **nama**, dan **alamat** menggunakan format *f-string* dengan **<7** agar jarak untuk titik dua sama. Pada bagian berikutnya, **tuple(nim)** mengubah string **NIM '71251233'** menjadi tuple per karakter, misalnya **('7', '1', '2', '5', '1', '2', '3', '3')**, lalu hasilnya dicetak ke layar. Setelah itu **nama.split()[0]** untuk mengambil nama depan saja dengan cara memecah nama berdasarkan spasi menjadi list kata-kata dan mengambil elemen pertama, yaitu **'Mikael'**. Nama depan tersebut kemudian diubah menjadi tuple karakter menggunakan **tuple(nama_depan_saja)** sehingga menghasilkan **('M', 'i', 'k', 'a', 'e', 'l')**, lalu dicetak ke layar. Terakhir, **nama.split()[::-1]** digunakan untuk membalik urutan kata pada nama **[::-1]**, sehingga urutannya menjadi **['Kuncoro', 'Adi', 'Satrio', 'Gratianus', 'Mikael']**. List tersebut kemudian diubah menjadi tuple menggunakan **tuple(nama_terbalik_list)** dan hasil akhirnya dicetak sebagai tuple nama terbalik.

SOAL 03



```
soal03.py > ...
1  nama_file = input("Enter a file name: ")
2
3  try:
4      handle = open(nama_file)
5  except FileNotFoundError:
6      print(f"File tidak ditemukan: {nama_file}")
7      exit()
8
9  distribusi_jam = dict()
10
11 for baris in handle:
12     baris = baris.rstrip()
13     if baris.startswith('From '):
14         kata = baris.split()
15         waktu = kata[5]
16         jam = waktu.split(':')[0]
17         distribusi_jam[jam] = distribusi_jam.get(jam, 0) + 1
18
19 list_jam = list(distribusi_jam.items())
20 list_jam.sort()
21
22 for jam, jumlah in list_jam:
23     print(f"{jam} {jumlah}")

PS > python -u "d:\New folder - Copy\Lap. PrakAlpro (A)\minggu13\soal03.py"
Enter a file name: mbox-short.txt
04 3
06 1
07 1
09 2
10 3
11 6
14 1
15 2
16 4
17 2
18 1
19 1
PS D:\New folder - Copy\Lap. PrakAlpro (A)\minggu13>
```

Gambar 2.7: Source code soal 03

Penjelasan:

Ini adalah program untuk menghitung distribusi jam dalam satu hari dimana ada pesan yang diterima dari setiap email yang masuk dengan menggunakan file mbox-short.txt sebagai datanya. Pertama Kode dimulai dengan **input()** untuk meminta nama file dari pengguna dan menyimpannya ke variabel **nama_file**. Lalu menggunakan blok **try-except** untuk membuka file dengan aman, jika file tidak ditemukan, program akan menampilkan pesan **error** lalu berhenti. Setelah itu membuat dictionary kosong **distribusi_jam** untuk menyimpan jumlah email berdasarkan jam pengiriman. File dibaca baris per baris, lalu setiap baris dibersihkan menggunakan **rstrip()**. Hanya baris yang diawali 'From ' yang diproses, kemudian baris tersebut dipecah menjadi list kata menggunakan **split()**. Waktu pengiriman diambil dari indeks ke-5, lalu bagian jam dipisahkan menggunakan **split(':')[0]**. Selanjutnya dictionary diperbarui dengan **.get(jam, 0) + 1** untuk menghitung berapa kali email muncul pada jam tertentu. Setelah semua data selesai diproses, dictionary diubah menjadi list tuple menggunakan **.items()**, diurutkan dengan **sort()**, lalu setiap pasangan jam dan jumlah dicetak ke layar menggunakan perulangan **for**.